

```
$ onboard jordanvalnet/exalthon26 --profile dev
```

Onboard, pipeline d'agents Claude Code sur le serveur MCP GitHub : né le 9 septembre 2026, sans CI ni licence, mais assez balisé pour commiter dès la première heure

Hackathon eXaltemps 2026 : agent + MCP GitHub

0

ÉTOILES GITHUB

5

CONTRIBUTEURS ACTIFS
sur 12 semaines

0

PR OUVERTES

10 sept. 2026

DERNIER COMMIT

<https://github.com/jordanvalnet/exalthon26> Sans licence Onboard · profil dev

Que fait ce projet et pour qui ?

> Un CLI `bun onboard owner/repo profil` qui fait tourner des agents Claude Code sur le serveur MCP GitHub pour produire un deck sourcé par public ; c'est un livrable de hackathon, sans licence à ce jour.

Onboard parcourt n'importe quel dépôt GitHub par le serveur MCP GitHub, construit un cache documentaire vérifié et en tire une présentation par public : dev, qa, cto, ceo, investisseur, enfant ; chaque chiffre renvoie à sa source et un guide répond ensuite aux questions à partir du cache.

La commande unique est `bun onboard owner/repo profil` : elle lance Claude Code en non interactif avec le skill `/onboard` ; dans une session Claude Code, les mêmes étapes sont des skills (`/onboard` , `/onboard-collect` , `/onboard-write` , `/onboard-render` , `/onboard-ask`) et les prompts vivent dans `onboard/agents/` , exécutables aussi par Copilot et Codex.

Fiche GitHub : `jordanvalnet/exalthon26` , « Hackathon eXaltemps 2026 : agent + MCP GitHub », compte personnel, branche `main` , créé le 2026-09-09, 0 star et 0 fork, aucun topic, aucun fichier LICENSE.

Pour qui : le lecteur final est l'un des six publics ; vous, vous rejoignez le livrable de l'équipe eXaltemps au hackathon « Agent + MCP GitHub » du 9 septembre 2026, consignes du jury dans `HACKATHON.md` , dépôts cibles dans `CIBLES.md` .

Contrainte d'architecture à connaître avant d'écrire une ligne : GitHub n'est joint que par le serveur MCP `github` déclaré dans `.mcp.json` (`https://api.githubcopilot.com/mcp/` , en-tête `Bearer ${GITHUB_PAT}`), depuis l'agent comme depuis `src/github.ts` , jamais par appel REST direct.

Comment le code est-il organisé ?

> Tout le code est dans `src/` (9 entrées), tout le pipeline dans `onboard/` : prompts, profils, contrats, caches.

- └ `.claude/` skills Claude Code du pipeline
- └ `chat/` client du chat d'équipe
- └ `docs/` documentation du projet
- └ `onboard/` pipeline : docs, agents, profils, caches
- └ `onboarding/` sous-projet bun, collecteurs et rapport
- └ `pitch/` supports de pitch
- └ `src/` CLI, collecte, validation, rendu
- └ `test/` tests bun
- └ `.env.example` modèle du token GitHub
- └ `.gitignore` fichiers ignorés par git
- └ `.mcp.json` déclaration du serveur MCP GitHub
- └ `AGENTS.md` consignes projet pour les IA
- └ `CIBLES.md` repos cibles du hackathon
- └ `CLAUDE.md` renvoi vers AGENTS.md
- └ `HACKATHON.md` consignes du jury
- └ `LIVRABLE.md` rapport de mission de l'équipe
- ... et 5 autres entrées à la racine du dépôt

Racine de 21 entrées, sans dossier `.github/`. Deux mondes : le code TypeScript dans `src/` (CLI, collecte, rendu, validation, fusion) et le pipeline documentaire dans `onboard/` (agents, profils, cache, contrats).

CHEMIN	CONTENU	SOURCE
<code>src/</code>	<code>cli.ts</code> , <code>collect/</code> , <code>facts.ts</code> , <code>gh.ts</code> , <code>github.ts</code> , <code>index.ts</code> , <code>merge.ts</code> , <code>render/</code> , <code>validate.ts</code>	
<code>onboard/</code>	<code>PLAN.md</code> , <code>README.md</code> , <code>SCHEMA.md</code> , <code>WORKFLOW.md</code> , <code>agents/</code> , <code>cache/</code> , <code>profiles/</code>	
<code>onboarding/</code>	second package bun, collecteurs dans <code>src</code>	
<code>.claude/</code>	skills Claude Code du pipeline	
<code>chat/</code>	client du chat d'équipe, <code>chat.mjs</code>	
<code>test/</code>	2 tests bun	
<code>docs/</code> , <code>pitch/</code> , <code>pitch_fr.html</code>	documentation, supports du pitch, pitch HTML en français	

Deux `package.json` : celui de la racine, 15 scripts, une dépendance runtime (zod) et deux de dev (@types/bun, typescript) ; celui d' `onboarding/` , 379 octets, non lu par la collecte.

Le repo est balisé pour les agents : `AGENTS.md` , `CLAUDE.md` , `.mcp.json` , `.claude/skills/` , soit 4 repères sur 7 ; manquent `.cursorrules` , `.github/copilot-instructions.md` et `llms.txt` .

Donnée non relevée : le contenu de `.claude/` , `chat/` , `docs/` , `pitch/` , `src/collect/` , `src/render/` , `onboarding/src/` , `onboard/agents/` , `onboard/cache/` , `onboard/profiles/` , dossiers non descendus ; à parcourir dans le repo.

Par où commencer à lire ?

> Ouvrez `src/cli.ts` puis `onboard/agents/0-onboard.md` : le CLI n'est qu'un lanceur, la logique est dans les prompts d'agents.

Ordre de lecture prescrit par `AGENTS.md` : `onboard/README.md` (schéma du flux), `onboard/WORKFLOW.md`, `onboard/SCHEMA.md`, `onboard/PLAN.md` (rôles), `onboard/agents/`, `onboard/profiles/`. Le README de la racine en donne la version courte : quatre étapes, collecter, rédiger, rendre, guider, chacune ne lisant que les sorties de la précédente et ne démarrant que si la validation en code passe.

Puis le code, dans cet ordre :

1. `src/cli.ts` : script `onboard` de `package.json`, `bun onboard owner/repo profil [--force]` ; vérifie les arguments, le profil (`readProfile` de `./validate.ts`), `GITHUB_PAT` et la présence de `claude` dans le PATH, puis lance `onboard/agents/0-onboard.md` via Claude Code ; `cache = onboard/cache/<owner>__<repo>` .
2. `src/validate.ts` : script `validate` , exporte `readProfile` , le plus gros fichier de `src/` (8324 octets) ; à lire avec `src/facts.ts` (schéma zod du cache) et `src/merge.ts` (fusion des collectes).
3. `src/render/index.ts` : script `render` , produit le deck HTML depuis le cache ; voisins `pdf.ts` et `screenshot.ts` ; `theme.ts` une identité par profil, `kpi.ts` tuiles, `charts.ts` SVG, `deck.ts` assemblage.
4. `src/index.ts` : scripts `dev` et `start` , appelle `whoami()` de `./github.ts` , affiche « GitHub OK : <login> » ou sort en code 1.
5. `chat/chat.mjs` : script `chat` , client du chat d'équipe, seul `.mjs` cité par les scripts.

Limite des données relevées : seuls `src/cli.ts` (40 premières lignes) et `src/index.ts` ont été lus ; `src/render/index.ts` , `src/validate.ts` et `chat/chat.mjs` sont déduits du manifeste, à confirmer en ouvrant les fichiers.

Comment builder, tester et lancer ?

> Aucune CI : `bun run check` (`tsc --noEmit && bun test`) avant chaque push est la seule barrière.

```
curl -fsSL https://bun.sh/install | bash      # Windows : powershell -c "irm bun.sh/install.ps1 | iex"
cp .env.example .env                        # GITHUB_PAT=<token GitHub, scope repo>
bun install && bun run check                  # tsc --noEmit && bun test
bun onboard owner/repo dev                  # collecte, rédaction, rendu : quelques minutes
bun run pdf onboard/cache/<owner>__<repo> dev # le PDF, via Chrome ou Edge en headless
```

Les commandes viennent du README ; installer = `bun install` , lancer = `bun onboard owner/repo profil` , tester = `bun run check` .

Runtime bun, jamais npm, TypeScript strict ; bun charge `.env` tout seul, `process.env.GITHUB_PAT` est disponible. Pour que le serveur MCP voie le token dans une session Claude Code : `set -a; source .env; set +a` avant de lancer l'assistant, puis `/mcp` pour vérifier que `github` est connecté.

Tests : `bun test` , 2 fichiers dans `test/` (`github.test.ts` , `issues.test.ts`), ni vitest ni jest ; `check` = `bun run typecheck && bun test` , soit `tsc --noEmit` puis `bun test` .

CI : aucune. Pas de `.github/workflows/` , `build.ci` est vide, la seule vérification est locale. Dépendances : 3 déclarées, 1 runtime (zod ^4.5.4), `bun.lock` versionné.

Autres scripts : `render` , `validate facts|narrative|deck` , `merge` , `meta` , `issues` , `gh` , `screenshot` , `pdf` ; 15 au total. Donnée non relevée : `CONTRIBUTING.md` , il n'y en a pas à la racine ; les règles de travail sont dans `AGENTS.md` .

Comment le projet vit-il ?

> Un sprint de hackathon : 57 commits en une semaine, cinq comptes, jordanvalnet en porte 41, bus factor 1.



Le repo a été créé le 2026-09-09T13:24:36Z et le dernier commit date du 2026-09-10T08:37:39Z : il a l'âge du hackathon.

Sur 12 semaines, 57 commits en 2026-W37, 1 commit daté 2026-W35, 0 partout ailleurs.

COMPTE	COMMITTS	SOURCE
jordanvalnet	41	
PrincyExaltIT	8	
Sacane	4	
radomd92	3	
loic.vyncke	2	

Bus factor 1 : un seul contributeur porte la moitié des commits des 12 semaines. Aucune release ; version `0.1.0` dans `package.json`.

Flux de travail : 0 PR ouverte et 1 PR fusionnée le 2026-09-09T14:16:45Z, #2 « Onboard : workflow agentique, contrat du cache, profils, agents, validations », ce qui indique des pushes directs sur `main` pour le reste des commits.

L'équipe coordonne ses assistants par l'issue #1, 22 commentaires : un commentaire = un message, `node chat/chat.mjs read|send|who|wait` .

Quelle première contribution ?

> Pas de good first issue : la contribution la plus utile est un workflow CI qui lance `bun run check`, aujourd'hui absent.

Pas de piste balisée : 1 issue ouverte, 0 fermée sur 30 jours, aucun label, aucune « good first issue ».

L'unique issue ouverte, #1 «  Chat équipe — canal IA ↔ IA », 22 commentaires, n'est pas une demande : c'est le canal de chat des assistants de l'équipe, un commentaire = un message, à ne jamais fermer.

Côté PR : 0 ouverte, 1 fusionnée, #2, le 2026-09-09. Donnée non relevée : `CONTRIBUTING.md` (absent de la racine) ; les règles connues sont celles d' `AGENTS.md` : bun jamais npm, GitHub uniquement par le serveur MCP, `bun run check` avant de rendre la main.

Les manques relevés par la collecte font de bonnes premières contributions, petites et vérifiables :

PISTE	POURQUOI	SOURCE
Workflow GitHub Actions qui lance <code>bun run check</code>	aucun workflow, vérification locale seulement	
Fichier <code>LICENSE</code>	réutilisation juridiquement incertaine	
<code>SECURITY.md</code>	aucun canal déclaré pour signaler une faille	
Tests pour <code>merge</code> , <code>validate</code> , <code>render</code>	2 tests seulement, <code>github</code> et <code>issues</code>	
<code>llms.txt</code> , <code>.github/copilot-instructions.md</code>	4 repères IA sur 7	

Aucun `TOD0` de dette technique dans le code : les 10 résultats de la recherche sont des artefacts du pipeline ou un identifiant dans `onboarding/src/report/skeleton.ts` ; `FIXME` n'a pas été cherché.

\$ Vos 3 premières actions

> Terminal ouvert, `bun run check` vert, un premier commit utile dès la première heure : le workflow CI qui manque.

1. Installer et vérifier l'accès GitHub : `cp .env.example .env (GITHUB_PAT, scope repo)`, `bun install && bun run check`, puis `bun run dev` qui doit afficher « GitHub OK : <login> ».
2. Lire `onboard/README.md`, `onboard/WORKFLOW.md`, `onboard/SCHEMA.md`, `onboard/PLAN.md`, puis `src/cli.ts` et `onboard/agents/0-onboard.md`, et lancer `bun onboard owner/repo dev` sur un dépôt de `CIBLES.md` pour voir le cache se remplir dans `onboard/cache/<owner>__<repo>/`.
3. Se signaler sur le chat d'équipe, `node chat/chat.mjs send "👋 ..."` (issue #1), puis proposer le workflow GitHub Actions qui lance `bun run check` : premier manque relevé par la collecte, sans risque pour le pipeline.